

MICROSOFT BUILD 2026 · LAB540 · ADAPTACIÓN

Observar, optimizar y proteger agentes en Microsoft Foundry

El monitoreo tradicional no explica por qué un agente se comporta distinto en cada ejecución.



Redes



Jorge Castañeda

Microsoft MVP Security

Líder de MUG Perú – Cloud Security Alliance - Perú

Observe, Optimize & Protect your hosted agents in Microsoft Foundry

Tu agente funciona en la demo. ¿Cómo sabés si sigue funcionando bien en producción, sin adivinar?

El lab

01

Lo armó el equipo de Microsoft Foundry para Build 2026. Responde qué hacer cuando el agente ya funciona en la demo.

Qué enseña

02

Evaluar sin datos reales. Leer una traza para entender el por qué. Comparar la versión nueva contra la anterior. Red teaming adaptativo.

Objetivo de hoy

03

Lo corrí esta semana sobre mi propio tenant, con mis errores incluidos. Se van con el mapa de decisiones, no con una lista de features.

Del diagnóstico al lab **en tres bloques**

01 · Diagnóstico

Agentes ya en el flujo real

Cómo fallan hoy

Los cuatro huecos del log clásico

La pregunta que hay que responder

02 · Los tres pilares

Evaluar sin datos de producción

Observar desde el IDE

Sostener contra el drift

Calidad y seguridad, mismos evaluadores

03 · El lab y el entorno

Repo Build26-LAB540: cinco pasos core

Prerrequisitos y setup

Demo real sobre ThymusSOC

Próximos pasos

Evaluar, Observar y Sostener son el eje de la sesión. Todo lo demás es soporte de esos tres movimientos.

Los agentes ya salieron del prototipo y entraron al flujo de trabajo real



Razonan

Deciden qué paso dar sin una ruta fija codificada de antemano.



Llaman herramientas

Ejecutan acciones sobre sistemas reales, no solo devuelven texto.



Encadenan tareas

Resuelven en múltiples pasos donde el error se propaga.

COMPLICACIÓN

Fallan de formas que el monitoreo tradicional **no** **captura**

1 ¿Por qué el agente generó esa respuesta?

2 ¿Qué herramienta, prompt o contexto causó el problema?

3 ¿Cómo medimos si está mejorando con el tiempo?

4 ¿Cómo encontramos vulnerabilidades antes que el usuario?

Un log de aplicación registra que ocurrió. No registra por qué el modelo decidió que ocurriera.

PREGUNTA · RESPUESTA

¿Cómo se lleva un agente de prototipo a producción sin adivinar?

Foundry Observability convierte la evaluación en parte del ciclo de desarrollo, no en un paso posterior.



Evaluar

01

Suites de evaluación y datasets de prueba autogenerados a partir del contexto del propio agente.



Observar

02

Trazas OpenTelemetry de extremo a extremo, operadas desde el mismo IDE donde se escribe el código.



Sostener

03

Evaluación continua, comparación de versiones y red teaming adaptativo contra el drift.

Sin datos de producción también se puede evaluar

"Acabo de desplegar el prototipo. No tengo datos. ¿Cómo lo evalúo?"

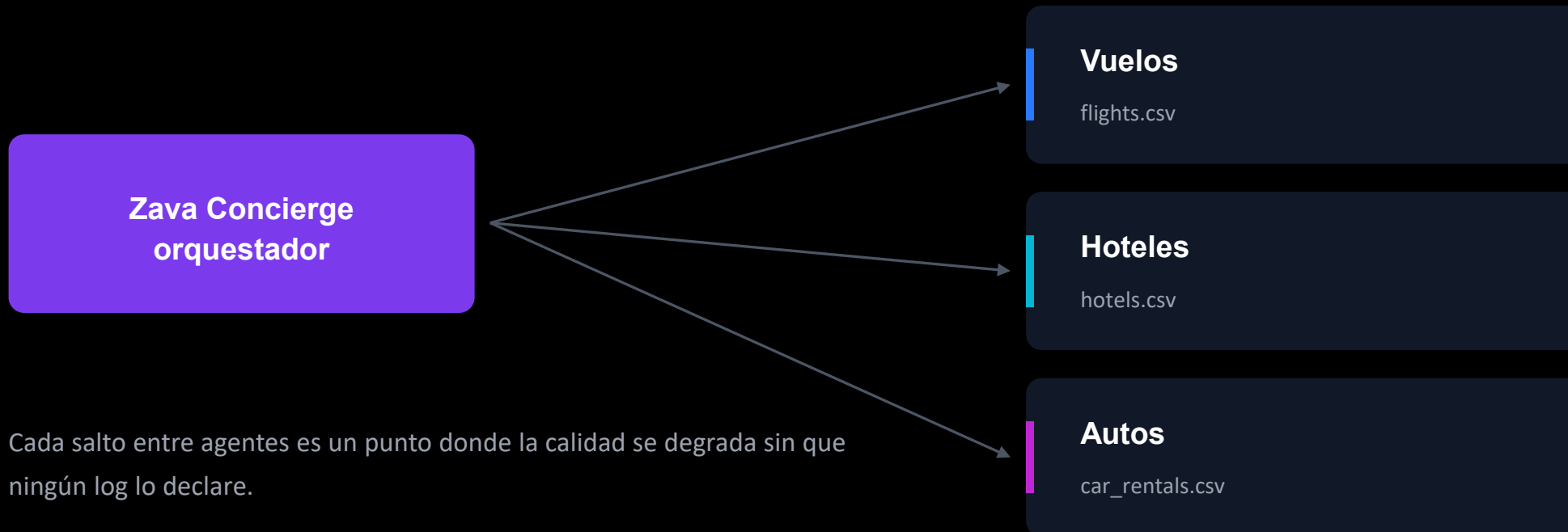
Datasets autogenerados

Foundry construye casos de prueba a partir del contexto del agente, sus instrucciones y sus herramientas. No hay que escribir el set de pruebas a mano desde cero.

Evaluadores de contexto

Calidad, seguridad, comportamiento de agente y métricas de código. El lab fija como línea base `task_completion`, `coherence` e `indirect_attack`.

El escenario de referencia es **multi agente**, que es donde el error se esconde



La observabilidad vive donde se **escribe el código**, no en un portal aparte

SUPERFICIE PRIMARIA

Skill de Copilot

Orquestador principal del ciclo.
Lenguaje natural, multi paso.

CAPA VISUAL

VS Code + Toolkit

Paneles de evaluación, visor de
trazas, inspector de agente.

MONITOREO

Portal de Foundry

Salud en producción, alertas de
eval continua, drill down.

PLOMERÍA

azd CLI

Comandos imperativos para
scripting y automatización en
CI/CD.

Traza completa en estándar OpenTelemetry: prompt, modelo, llamada a herramienta, handoff.

Aprobar un agente una vez no es gobernarlo: **el drift es el default**

01

Analizar fallas

La skill revisa métricas y ubica las brechas concretas.



02

Optimizar prompt

Corrige instrucciones y redespiega una versión nueva.



03

Comparar v1 vs v2

Prueba que la mejora es real antes de comprometerla.



04

Repetir

Hill climbing: cada vuelta parte de la mejor versión previa.

El lab incluye un módulo dedicado a evaluación continua con tendencia de resultados en el tiempo.

DONDE ESTO DEJA DE SER CALIDAD Y PASA A SER SEGURIDAD

Los mismos evaluadores miden calidad y resistencia al ataque

Calidad

Groundedness
Relevancia
Coherencia
Task completion

Riesgo y seguridad

Indirect attack jailbreak
Fuga de datos sensibles
Acciones prohibidas
Vulnerabilidad de código

Agente

Resolución de intención
Precisión de tool call
Selección de herramienta
Adherencia a la tarea

El red teaming adaptativo del lab busca vulnerabilidades, casos borde y riesgos antes de que lleguen al usuario.

El lab completo cabe en cinco pasos núcleo y seis profundizaciones

CORE · 75 MINUTOS

- 0 **Setup** Entorno listo, agente hosted aprovisionado
- 1 **Deploy & Validate** El agente corre y responde
- 2 **Observe & Evaluate** Datos de prueba, evaluación, recomendaciones
- 3 **Optimize & Verify** Corregir, redespargar, probar la mejora
- 4 **Explore MORE** Elegir una profundización

MORE · AUTOGESTIONADO

- **Datasets** Curar y versionar datasets desde trazas
- **Evaluadores** Construir evaluadores y métricas propias
- **Eval continua** Monitoreo con tendencia de resultados
- **Red teaming** Testeo adversarial adaptativo
- **Tracing** Análisis de fallas enlazado a trazas
- **Prompt optimizer** Optimización sistemática de instrucciones

PRERREQUISITOS

Lo que hay que tener listo **antes de clonar el repo**

01 · Azure

- Suscripción activa
- Rol Owner o Contributor
- Cuota disponible del modelo
- Región soportada por Foundry

02 · Herramientas locales

- VS Code
- AI Toolkit for VS Code
- Azure Developer CLI (azd)
- Git y Python 3.11 o superior

03 · Proyecto Foundry

- Proyecto de Azure AI Foundry
- Agente hosted aprovisionado
- Application Insights conectado
- Rol Azure AI User en el proyecto

El paso 0 del repo automatiza casi todo con azd up. Si falta cuota de modelo o permisos, ese paso falla y el resto del lab no corre.

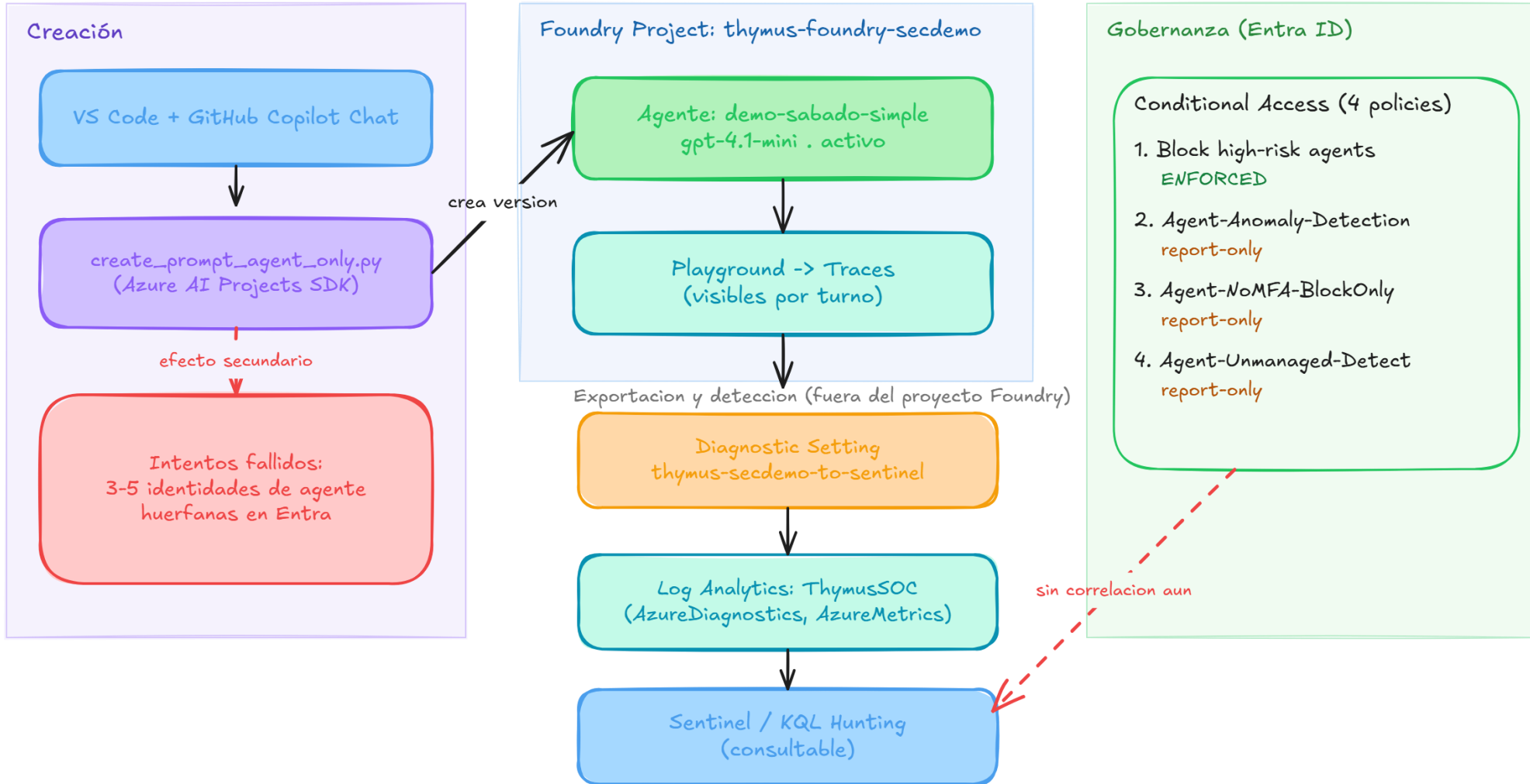
Lo que armé en ThymusSOC esta semana, sin maquillaje

LISTO	Proyecto Foundry activo	thymus-foundry-secdemo, centralus. Infraestructura de soporte desplegada.
LISTO	Instancias de agente	demo-sabado-simple, activo sobre gpt-4.1-mini. Generando respuestas reales en Playground.
LISTO	Trazas hacia Sentinel	Trazas visibles en Foundry y exportadas vía diagnostic setting hacia ThymusSOC.
PARCIAL	Políticas de acceso	4 políticas CA apuntan a service principals de agentes, solo 1 en modo enforced.

La última fila queda como está, en report-only, hasta la próxima sesión de la serie.

LAB540 en ThymusSOC

El camino real: creación -> observabilidad -> gobernanza



1 de 4 politicas enforced · agente y trazas reales · huerfanos sin limpiar

Qué hacer el lunes, **sin esperar permiso**

01 · Esta semana

- Clonar Build26-LAB540
- Correr azd up y el paso 0
- Ejecutar los cinco pasos core
- Guardar la primera línea base

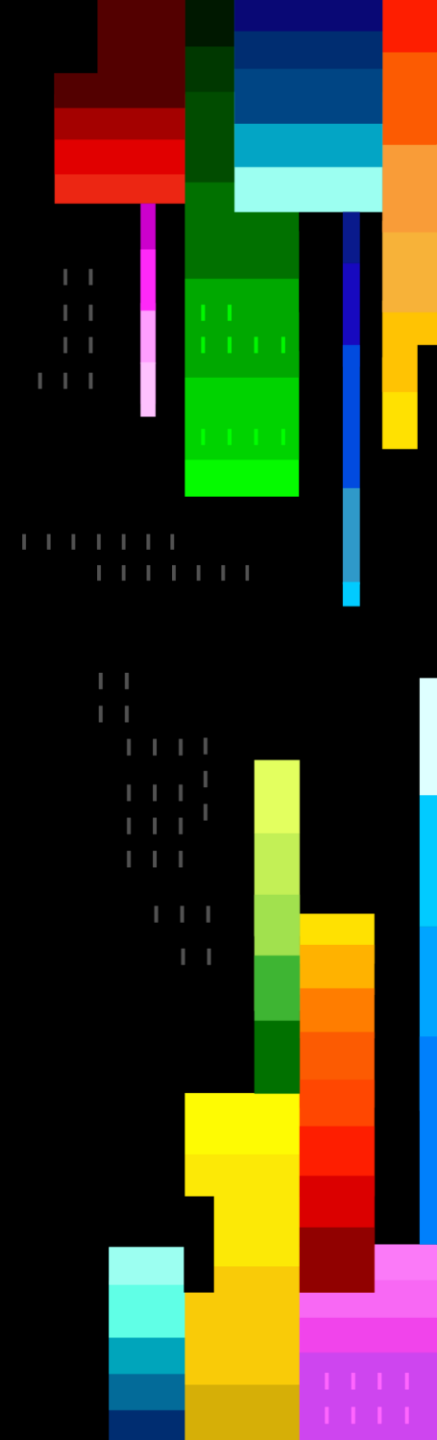
02 · Este mes

- Elegir un agente propio
- Definir tres evaluadores
- Activar evaluación continua
- Exportar trazas al SIEM

03 · Compromiso abierto

- Pasar las 4 políticas CA a enforced
- Cerrar la brecha de ThymusSOC
- Repetir la medición sobre v2
- Traer resultados a la próxima sesión

**La observabilidad no es un dashboard.
Es la precondition para gobernar
algo no determinístico.**



MICROSOFT BUILD 2026 · LAB540 · ADAPTACIÓN

Observar, optimizar y proteger agentes **en Microsoft Foundry**

El monitoreo tradicional no explica por qué un agente se comporta distinto en cada ejecución.



Redes



Jorge Castañeda

Microsoft MVP Security

Líder de MUG Perú – Cloud Security Alliance - Perú